

Day 4: State and Interactivity

Complete Lab Steps and Code

This guide takes the existing TaskBoard from its Day 3 static board through all three Day 4 labs. It follows the finished checkpoints in this repository. Each lab builds on the previous one.

Before you start

Requirements: Node.js 24 LTS and npm.

In a terminal, start in the repository folder and run:

```
cd Projects/Day04_State_and_Interactivity/starter/taskboard
npm install
npm run dev
```

Open the local URL printed by Vite. Keep this terminal running while you work. In another terminal, use these checks whenever a checkpoint is complete:

```
npm run lint
npm run build
```

The starter already contains the Day 3 TaskBoard, product catalogue, `products.js`, `formatPrice.js`, and base styles. The code below creates or changes only the files needed for Day 4. Keep the existing Day 3 CSS and append the Day 4 CSS shown in each lab.

The `solution/lab-4.1-counter-toggle-accordion/`, `solution/lab-4.2-shopping-cart/`, and `solution/lab-4.3-taskboard/` folders are complete runnable checkpoints if you need to compare your work or catch up.

Lab 4.1: Counter, Theme Toggle, Accordion

Goal: Practise `useState`, event handlers, updater functions and lifting state.

Steps

1. Create `src/components/Counter.jsx`. Add Plus, Minus, Plus 5 and Reset buttons.
2. Use an updater function for Plus and Minus. Clamp the count at zero.
3. Create `src/components/ThemeToggle.jsx`. Store whether dark mode is active and toggle the wrapper class.
4. Create `src/components/Accordion.jsx`. Keep the open item IDs in the parent component so every item can open independently.
5. Derive `allOpen` from the open IDs and add a Show all / Hide all button.
6. Import these three components into `src/App.jsx` and render them below the board in a temporary practice section.
7. Try each control and inspect component state in React DevTools.
8. Run lint and build. Confirm the counter cannot go below zero and the accordion items can be controlled independently.
9. Commit the completed checkpoint if you are using version control for the course.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  function handlePlus() {
    setCount((current) => current + 1);
  }

  function handleMinus() {
    setCount((current) => Math.max(0, current - 1));
  }

  function handlePlusFive() {
    for (let index = 0; index < 5; index++) {
      setCount((current) => current + 1);
    }
  }

  return (
    <div className="counter">
      <p>Count: {count}</p>
      <button onClick={handleMinus}>Minus</button>
      <button onClick={handlePlus}>Plus</button>
      <button onClick={handlePlusFive}>Plus 5</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}

export default Counter;
```

The updater form receives the latest pending value. That is why five calls inside the loop add five rather than repeatedly using the same render's `count` snapshot.

src/components/ThemeToggle.jsx

```
import { useState } from "react";

function ThemeToggle() {
  const [isDark, setIsDark] = useState(false);

  return (
    <div className={isDark ? "dark" : "light"}>
      <p>The current theme is {isDark ? "dark" : "light"}.</p>
      <button onClick={() => setIsDark((current) => !current)}>
        Toggle theme
      </button>
    </div>
  );
}

export default ThemeToggle;
```

src/components/Accordion.jsx

```
import { useState } from "react";

const items = [
  { id: "state", title: "What is state?", body: "Data a component remembers between renders." },
  { id: "props", title: "What are props?", body: "Read-only inputs passed in by the parent." },
  { id: "events", title: "What is an event handler?", body: "A function React calls when something happens." },
];

function AccordionItem({ title, isOpen, onToggle, children }) {
  return (
    <div className="accordion-item">
      <button onClick={onToggle} aria-expanded={isOpen}>
        {title}
      </button>
      {isOpen && <p>{children}</p>}
    </div>
  );
}

function Accordion() {
  const [openIds, setOpenIds] = useState([]);
  const allOpen = openIds.length === items.length;

  function handleToggle(id) {
    setOpenIds((previousIds) =>
      previousIds.includes(id)
        ? previousIds.filter((openId) => openId !== id)
        : [...previousIds, id]
    );
  }

  function handleToggleAll() {
    setOpenIds(allOpen ? [] : items.map((item) => item.id));
  }

  return (
    <section>
      <button onClick={handleToggleAll}>
        {allOpen ? "Hide all" : "Show all"}
      </button>
      {items.map((item) => (
        <AccordionItem
          key={item.id}
          title={item.title}
          isOpen={openIds.includes(item.id)}
          onToggle={handleToggle(item.id)}
        />
      ))}
    </section>
  );
}
```

```

        isOpen={openIds.includes(item.id)}
        onToggle={() => handleToggle(item.id)}
      >
        {item.body}
      </AccordionItem>
    )}
  </section>
);
}

export default Accordion;

```

Update `src/App.jsx` for Lab 4.1

Keep the existing Header and Board. Add these imports and render the temporary practice section underneath them:

```

import './App.css';
import Header from './components/Header.jsx';
import Board from './components/Board.jsx';
import Counter from './components/Counter.jsx';
import ThemeToggle from './components/ThemeToggle.jsx';
import Accordion from './components/Accordion.jsx';
import { tasks } from './data/tasks.js';

function App() {
  return (
    <>
      <Header tasks={tasks} />
      <Board tasks={tasks} />

      <section className="practice">
        <h2>Lab 4.1 practice</h2>
        <Counter />
        <ThemeToggle />
        <Accordion />
      </section>
    </>
  );
}

export default App;

```

Append to `src/App.css` for Lab 4.1

```

.practice {
  margin-top: 32px;
  padding-top: 16px;
  border-top: 1px solid #e5e7eb;
}

.practice button {
  font: inherit;
  margin: 0 8px 8px 0;
}

.light,
.dark {
  padding: 12px;
  border-radius: 8px;
  margin: 16px 0;
}

```

```
.light {  
  background: #ffffff;  
  color: #1f2937;  
}  
  
.dark {  
  background: #1f2937;  
  color: #f9fafb;  
}  
  
.accordion-item {  
  margin-top: 8px;  
}  
  
.accordion-item p {  
  margin: 4px 0 0;  
}
```

Done when: the counter never becomes negative, accordion items open independently, and Show all / Hide all controls every item.

Lab 4.2: Shopping Cart

Goal: Manage an array of cart items immutably and calculate totals from state.

Steps

1. Reuse the Lab 3.2 product catalogue in `src/components/catalogue/products.js`. The starter already has the products and price-formatting helper.
2. Pass an `onAddToCart` callback through `ProductGrid` to each `ProductCard`.
3. Add an Add to cart button to each product. Disable it when the product is out of stock.
4. Create `src/components/catalogue/Shop.jsx`. Keep cart state here because `Shop` is the common parent of the catalogue and cart.
5. Store cart lines as `{ productId, quantity }`. Increment an existing line rather than adding a duplicate.
6. Create `src/components/catalogue/Cart.jsx`. Add plus, minus and Remove controls; remove lines once their quantity reaches zero.
7. Calculate item count and total during render. Do not store derived totals in state.
8. Render `<Shop />` below the board temporarily. Run lint and build, then test duplicate adds, quantity changes, removal and out-of-stock behavior.
9. Commit the completed checkpoint if you are using version control for the course.

`src/components/catalogue/Shop.jsx`

```
import { useState } from "react";
import { products } from "../products.js";
import ProductGrid from "../ProductGrid.jsx";
import Cart from "../Cart.jsx";

function Shop() {
  const [cart, setCart] = useState([]);

  function handleAddToCart(id) {
    setCart((previousCart) => {
      const existingItem = previousCart.find((item) => item.productId === id);
      if (existingItem) {
        return previousCart.map((item) =>
          item.productId === id
            ? { ...item, quantity: item.quantity + 1 }
            : item
        );
      }
      return [...previousCart, { productId: id, quantity: 1 }];
    });
  }

  function handleChangeQuantity(id, amount) {
    setCart((previousCart) =>
      previousCart
        .map((item) =>
          item.productId === id
            ? { ...item, quantity: item.quantity + amount }
            : item
        )
        .filter((item) => item.quantity > 0)
    );
  }
}
```

```

function handleRemove(id) {
  setCart((previousCart) =>
    previousCart.filter((item) => item.productId !== id)
  );
}

return (
  <div className="shop">
    <ProductGrid products={products} onAddToCart={handleAddToCart} />
    <Cart
      cart={cart}
      products={products}
      onChangeQuantity={handleChangeQuantity}
      onRemove={handleRemove}
    />
  </div>
);
}

export default Shop;

```

src/components/catalogue/ProductGrid.jsx

```

import ProductCard from "../ProductCard.jsx";

function ProductGrid({ products, onAddToCart }) {
  if (products.length === 0) return <p>No products</p>;

  return (
    <div className="grid">
      {products.map((product) => (
        <ProductCard
          key={product.id}
          {...product}
          onAddToCart={onAddToCart}
        />
      ))}
    </div>
  );
}

export default ProductGrid;

```

src/components/catalogue/ProductCard.jsx

```

import { formatPrice } from "../formatPrice.js";

function ProductCard({ id, name, price, rating, inStock, onAddToCart }) {
  return (
    <article className="card product-card">
      <h3>{name}</h3>
      <p>{formatPrice(price)}</p>
      {rating > 0 && <p>{"â~".repeat(rating)}</p>}
      {!inStock && <span className="badge">Out of stock</span>}
      <button onClick={() => onAddToCart(id)} disabled={!inStock}>
        Add to cart
      </button>
    </article>
  );
}

export default ProductCard;

```

src/components/catalogue/Cart.jsx

```
import { formatPrice } from "../formatPrice.js";

function Cart({ cart, products, onChangeQuantity, onRemove }) {
  const itemCount = cart.reduce(
    (totalItems, item) => totalItems + item.quantity,
    0
  );

  const total = cart.reduce((sum, item) => {
    const product = products.find((currentProduct) => currentProduct.id === item.productId);
    return sum + product.price * item.quantity;
  }, 0);

  if (cart.length === 0) {
    return (
      <aside className="cart">
        <p>Your cart is empty</p>
      </aside>
    );
  }

  return (
    <aside className="cart">
      <h2>Cart ({itemCount} items)</h2>
      <ul>
        {cart.map((item) => {
          const product = products.find(
            (currentProduct) => currentProduct.id === item.productId
          );
          return (
            <li key={item.productId}>
              {product.name} x {item.quantity}
              <button onClick={() => onChangeQuantity(item.productId, -1)}>
                -
              </button>
              <button onClick={() => onChangeQuantity(item.productId, 1)}>
                +
              </button>
              <button onClick={() => onRemove(item.productId)}>Remove</button>
            </li>
          );
        })}
      </ul>
      <p>Total: {formatPrice(total)}</p>
    </aside>
  );
}

export default Cart;
```

The product data and `formatPrice.js` are provided by the Day 3 catalogue. The formatter uses South African rand (`en-ZA`, `ZAR`).

Update `src/App.jsx` for Lab 4.2

Remove the Lab 4.1 imports and practice section. Render the shop temporarily below the static board:

```
import "../App.css";
import Header from "../components/Header.jsx";
import Board from "../components/Board.jsx";
import Shop from "../components/catalogue/Shop.jsx";
import { tasks } from "../data/tasks.js";
```

```
function App() {
  return (
    <>
      <Header tasks={tasks} />
      <Board tasks={tasks} />

      <section className="practice">
        <h2>Lab 4.2 shop</h2>
        <Shop />
      </section>
    </>
  );
}

export default App;
```

Append to `src/App.css` for Lab 4.2

```
.shop {
  display: grid;
  grid-template-columns: 2fr 1fr;
  gap: 16px;
  align-items: start;
}

.product-card button {
  display: block;
  margin-top: 8px;
}

.cart {
  background: #ffffff;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 16px;
}

.cart h2 {
  margin-top: 0;
  font-size: 1.125rem;
}

.cart ul {
  padding-left: 16px;
}

.cart li button {
  margin: 0 0 4px 6px;
}
```

Done when: there are no duplicate cart lines, quantities cannot go below one, and totals always match the cart. Totals are derived, not state.

Lab 4.3: Interactive TaskBoard

Goal: Add, move, rename and delete tasks with state owned by `App`.

Steps

1. Import `useState` and rename the data import to `initialTasks`. Put the task array into state in `App`.
2. Add immutable handlers for adding, changing status, renaming and deleting tasks.
3. Create `src/components/AddTaskForm.jsx` with controlled title, assignee and points fields. Require a title of at least three characters and convert points to a number.
4. Add a status dropdown to each task card. The three values are `todo`, `doing` and `done`.
5. Add inline editing. Keep edit mode and the draft title as local `TaskCard` state.
6. Add Delete with a confirmation prompt.
7. Pass handlers from `App` through `Board` and `Column` to each `TaskCard`.
8. Change `Header` to derive and display the number of done tasks.
9. Remove the temporary Shop section from `App` and test every action.
10. Commit and push if your course uses version control, then run lint and build and verify counts update immediately.

src/App.jsx

```
import { useState } from "react";
import "./App.css";
import Header from "./components/Header.jsx";
import AddTaskForm from "./components/AddTaskForm.jsx";
import Board from "./components/Board.jsx";
import { tasks as initialTasks } from "./data/tasks.js";

function App() {
  const [tasks, setTasks] = useState(initialTasks);

  function handleAdd(newTask) {
    const id = crypto.randomUUID();
    setTasks((previousTasks) => [
      ...previousTasks,
      { ...newTask, id, status: "todo" },
    ]);
  }

  function handleStatusChange(id, status) {
    setTasks((previousTasks) =>
      previousTasks.map((task) =>
        task.id === id ? { ...task, status } : task
      )
    );
  }

  function handleRename(id, title) {
    setTasks((previousTasks) =>
      previousTasks.map((task) =>
        task.id === id ? { ...task, title } : task
      )
    );
  }

  function handleDelete(id) {
```

```

    setTasks((previousTasks) =>
      previousTasks.filter((task) => task.id !== id)
    );
  }

  return (
    <>
      <Header tasks={tasks} />
      <AddTaskForm onAdd={handleAdd} />
      <Board
        tasks={tasks}
        onStatusChange={handleStatusChange}
        onRename={handleRename}
        onDelete={handleDelete}
      />
    </>
  );
}

export default App;

```

src/components/AddTaskForm.jsx

```

import { useState } from "react";

const emptyForm = { title: "", assignee: "", points: 1 };

function AddTaskForm({ onAdd }) {
  const [form, setForm] = useState(emptyForm);
  const [error, setError] = useState("");

  function handleChange(event) {
    const { name, value } = event.target;
    setForm((previousForm) => ({ ...previousForm, [name]: value }));
  }

  function handleSubmit(event) {
    event.preventDefault();
    const title = form.title.trim();
    if (title.length < 3) {
      setError("Title needs 3+ characters.");
      return;
    }

    setError("");
    onAdd({
      title,
      assignee: form.assignee.trim(),
      points: Number(form.points),
    });
    setForm(emptyForm);
  }

  return (
    <form className="add-task-form" onSubmit={handleSubmit}>
      <label htmlFor="title">Title</label>
      <input
        id="title"
        name="title"
        value={form.title}
        aria-invalid={Boolean(error)}
        onChange={handleChange}
      />

      <label htmlFor="assignee">Assignee</label>
      <input

```

```

        id="assignee"
        name="assignee"
        value={form.assignee}
        onChange={handleChange}
      />

      <label htmlFor="points">Points</label>
      <input
        id="points"
        name="points"
        type="number"
        min="1"
        value={form.points}
        onChange={handleChange}
      />

      {error && <p role="alert">{error}</p>}
      <button type="submit">Add task</button>
    </form>
  );
}

export default AddTaskForm;

```

src/components/Board.jsx

```

import Column from "../Column.jsx";

const columns = [
  ["todo", "To do"],
  ["doing", "In progress"],
  ["done", "Done"]
];

function Board({ tasks, onStatusChange, onRename, onDelete }) {
  return (
    <div className="board">
      {columns.map(([status, heading]) => (
        <Column
          key={status}
          heading={heading}
          tasks={tasks.filter((task) => task.status === status)}
          onStatusChange={onStatusChange}
          onRename={onRename}
          onDelete={onDelete}
        />
      ))}
    </div>
  );
}

export default Board;

```

src/components/Column.jsx

```

import TaskCard from "../TaskCard.jsx";

function Column({ heading, tasks, onStatusChange, onRename, onDelete }) {
  return (
    <section className="column">
      <h2>{heading} ({tasks.length})</h2>
      {tasks.length === 0 ? (
        <p>Nothing here yet</p>
      ) : (
        <ul>
          {tasks.map((task) => (
            <li key={task.id}>

```

```

        <TaskCard
          task={task}
          onStatusChange={onStatusChange}
          onRename={onRename}
          onDelete={onDelete}
        />
      </li>
    )}
  </ul>
)
</section>
);
}

export default Column;

```

src/components/TaskCard.jsx

```

import { useState } from "react";

function TaskCard({ task, onStatusChange, onRename, onDelete }) {
  const [isEditing, setIsEditing] = useState(false);
  const [draft, setDraft] = useState(task.title);

  function handleSave() {
    const title = draft.trim();
    if (title === "") return;
    onRename(task.id, title);
    setIsEditing(false);
  }

  function handleDeleteClick() {
    if (window.confirm(`Delete "${task.title}"?`)) {
      onDelete(task.id);
    }
  }

  return (
    <article className="card">
      {isEditing ? (
        <>
          <input
            aria-label="Task title"
            value={draft}
            onChange={(event) => setDraft(event.target.value)}
          />
          <button onClick={handleSave}>Save</button>
        </>
      ) : (
        <>
          <h3>{task.title}</h3>
          <button onClick={() => setIsEditing(true)}>Edit</button>
        </>
      )}

      {task.assignee && <p>Assigned to {task.assignee}</p>}
      {task.points > 0 && <span>{task.points} pts</span>}

      <select
        aria-label="Status"
        value={task.status}
        onChange={(event) => onStatusChange(task.id, event.target.value)}
      >
        <option value="todo">To do</option>
        <option value="doing">In progress</option>
        <option value="done">Done</option>
      </select>
    </article>
  );
}

```

```

        </select>

        <button onClick={handleDeleteClick}>Delete</button>
      </article>
    );
  }

  export default TaskCard;

```

src/components/Header.jsx

```

function Header({ tasks }) {
  const doneCount = tasks.filter((task) => task.status === "done").length;

  return (
    <header className="header">
      <h1>TaskBoard</h1>
      <p>{doneCount} of {tasks.length} done</p>
    </header>
  );
}

export default Header;

```

Append to src/App.css for Lab 4.3

```

.add-task-form {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 8px;
  margin-bottom: 16px;
}

.add-task-form input {
  font: inherit;
  padding: 4px 8px;
}

.add-task-form input[type="number"] {
  width: 64px;
}

.add-task-form p[role="alert"] {
  flex-basis: 100%;
  margin: 0;
  color: #991b1b;
}

.add-task-form button,
.column .card button,
.column .card select {
  font: inherit;
}

.column .card button,
.column .card select {
  margin: 4px 6px 0 0;
}

.column .card > span {
  display: block;
  font-size: 0.875rem;
}

```

```
font-weight: 600;  
color: #0f766e;  
}
```

Done when: tasks can be added, moved between columns, renamed and deleted; counts update immediately; and all array updates use `map`, `filter` or spread rather than mutation.

Final Check

From the TaskBoard project folder:

```
npm run lint  
npm run build
```

Run the app with `npm run dev` and test these actions:

- Add a task with a valid title; try an invalid short title too.
- Move a task through To do, In progress and Done. Check the column and header counts.
- Rename a task and save it.
- Delete a task and confirm the prompt.
- Refresh the page. The board returns to its original sample data; persistence is covered on Day 5.

The final Lab 4.3 checkpoint intentionally removes the temporary Lab 4.1 practice controls and Lab 4.2 shop from `App.jsx`. Their component files can remain in the project; the final screen is the interactive TaskBoard.